



RAJALAKSHMI ENGINEERING COLLEGE

PROJECT REPORT

AI Data Detective - Autonomous Data Quality Root Cause

Investigation Agent

Project Review 2

AD23731 - Foundation of Agentic AI

Subject In-charge: Dr. Priyadharsini Ravisankar

DOMAIN:	Agentic AI, Data Engineering, Data Quality Management
AGENT FRAMEWORK:	LangGraph, LangChain, RAG, LiteLLM
ML MODELS:	Groq(LLM) Gemini (LLM), Rule-Based Statistical Analysis

Jegadeeswaran D (231801069)

Karthick S (231801079)

Koushal V (231801084)

TABLE OF CONTENTS

S. No.	Title	Page No.
	Project Overview	3
1	Functional Prototype	4
2	Agent Orchestration Workflow Implementation	4
3	Tool Integrations	5
4	Agentic Rag Implementation	6
5	Memory And State Management	6
6	Preliminary Evaluation Results	6
7	Comparison With Baseline Solution	8

REVIEW 2

OBSIDIANDQ AN AGENTIC DATA-QUALITY INVESTIGATION AND REMEDIATION SYSTEM

Project Overview

ObsidianDQ is a local hybrid agentic framework for autonomous data quality investigation and remediation. It targets a specific gap in existing data pipelines: multi-stage ELT/ETL systems can silently propagate data quality problems (missing values, negative numeric fields, invalid dates, duplicate records, broken join keys) across downstream stages without an immediate failure — the corruption instead surfaces later as bad dashboards, reports, or ML model inputs.

Problem being addressed (from the report's problem statement):

1. Traditional DQ tools (Great Expectations, Soda, Monte Carlo) are **passive and alert-only** — they detect rule violations but cannot investigate upstream lineage, dynamically query data, or propose a repair.
2. Naive LLM-based remediation agents solve the investigation gap but introduce **uncontrolled risk** — hallucinated schemas, invalid SQL, and direct database write access.
3. Both approaches generally **lack truthful, closed-loop verification** — success is often assumed rather than re-checked after remediation.

Core design principle: separate *reasoning* from *execution*. LLM agents are restricted to read-only diagnostic tools and produce structured proposals; all actual data scanning, lineage traversal, SQL transformation, quarantine writes, and verification are handled by deterministic Python/AST components. The LLM has "eyes but no hands."

System layers (four, as defined in the architecture):

1. Deterministic data quality detection
2. Lineage-based root-cause analysis (DAG traversal)
3. Stateful multi-agent investigation and planning (LangGraph)
4. Deterministic remediation and verification

Primary objectives set for the project:

- Build a hybrid execution graph where deterministic engines retain exclusive write authority, and LLM agents act only as investigators/planners/critics.
- Implement lineage-based RCA to trace failures to their earliest upstream origin and estimate downstream blast radius.
- Provide AST-grounded (not free-text LLM-generated) SQL healing via sqlglot.
- Enforce Human-in-the-Loop governance for high-risk, low-confidence, or unresolved cases.
- Guarantee truthful post-remediation verification rather than assumed success.
- Maintain audit-ready, persistent observability of every run, tool call, and evaluation outcome via DuckDB.

This framing sets up why the seven deliverables below matter: each one maps to a specific objective above (e.g., orchestration → HITL enforcement, tool integration → the read-only boundary, evaluation

→ the truthful-verification objective, baseline comparison → the "passive tools vs. unsafe agents vs. hybrid" problem statement).

1. FUNCTIONAL PROTOTYPE

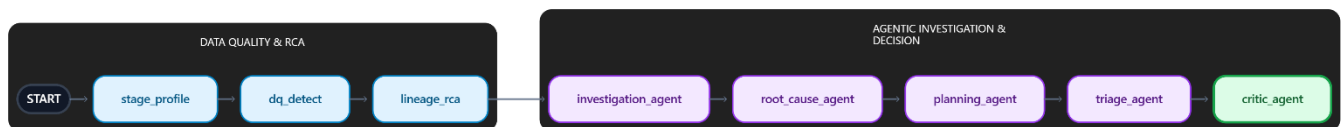
ObsidianDQ is implemented as a full-stack, locally-runnable system with four coordinated layers:

- **Frontend:** Next.js 14 (App Router) + React + Tailwind CSS + Framer Motion, with a guided three-step flow — UploadStep → ConfirmationTicketStep → PipelineRunScreen → RunConsole. The Run Console exposes an Overview, Issues, Lineage DAG (via @xyflow/react), data preview/profiling, an Actions/Approval gate, and SQL diff/technical details.
- **Backend:** FastAPI exposing GET /api/health, POST /api/pipeline/upload, POST /api/pipeline/run, POST /api/pipeline/approve, POST /api/pipeline/quarantine.
- **Persistence:** An embedded, thread-safe DuckDB database (data/obsidiandq.duckdb) with six tables — runs, incidents, dq_results, rca_evidence, remediation_results, evaluation_records — replacing an earlier JSONL-based store.
- **Default demo inputs:** stg_orders.parquet, raw_customers.csv, fct_sales.sql, lineage.json.

The system runs end-to-end: upload/preset → deterministic detection → lineage RCA → agentic investigation/planning/triage/critique → (optional) human approval → deterministic SQL healing and quarantine → verification → formatted telemetry back to the UI. This constitutes a working prototype rather than a design mockup — every stage listed has a corresponding implemented module (e.g., src/agent/nodes/dq_detect.py).

2. AGENT ORCHESTRATION WORKFLOW IMPLEMENTATION

Orchestration is built on **LangGraph**, coordinating a fixed graph:



From critic_agent, routing branches five ways:

- **Approved** → sql_healer
- **Revision required (first retry)** → back to investigation_agent
- **Review required** → human_review_queue
- **No issues found** → guardrails
- **Escalation route** → investigation_agent

Key orchestration properties:

- A **bounded critic loop** (max one retry) prevents indefinite re-investigation.
- **LangGraph interrupt_before=["human_review_queue"]** pauses execution mid-graph and checkpoints state via MemorySaver, keyed on run_id.
- Approval/rejection resumes or terminates the graph (APPROVAL_REJECTED → END on rejection, with no writes performed).
- After remediation, verify_agent re-runs detection on the cleaned artifact and routes to either guardrails → END (pass) or verification_recovery → VERIFICATION_FAILED → guardrails → END.

This is a genuine multi-agent, stateful, interruptible workflow — not a single-shot LLM call — with five distinct agent roles (Investigation, Root-Cause, Planning, Triage, Critic) each consuming and producing structured AgentState fields.

3. TOOL INTEGRATIONS

Investigation Agent tools (read-only, via Groq tool calling when configured):

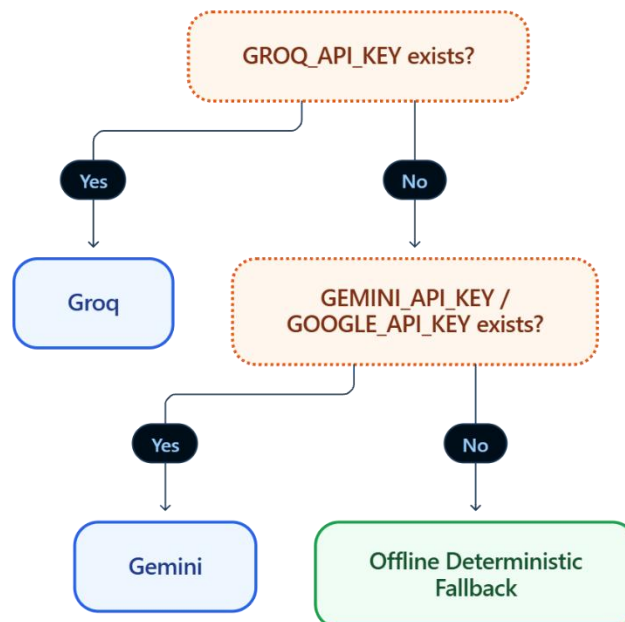
Tool	Function
get_sample_rows	Reads CSV/Parquet, returns up to 5 matching offending rows
get_column_stats	Returns data type, null count, distinct count, row count
get_lineage_context	Returns direct upstream nodes, ancestors, descendants
search_similar_incidents	Searches historical incident records in DuckDB

The Investigation Agent has **no** data-writing or SQL-writing tool — a deliberate safety boundary. If Groq is unavailable/fails, the system falls back to deterministic evidence (sample rows, similar incidents, lineage context) rather than blocking.

External service/library integrations:

- **Groq** (preferred LLM provider, default model openai/gpt-oss-20b) — used for Investigation Agent tool calling, with retry handling for HTTP 429 rate limits.
- **Gemini** (gemini-2.5-flash) — fallback provider for later text-oriented agents (not used for function calling).
- **sqlglot** — AST-based SQL parsing/repair (DuckDB dialect) for the SQL healer.
- **DuckDB** — embedded analytical persistence.

Provider selection logic:



4. AGENTIC RAG IMPLEMENTATION

Be precise here for your report, since this is the deliverable most easily overstated. What the documents actually describe:

- The Investigation Agent calls `search_similar_incidents(column, rule)`, which **queries historical incident records stored in DuckDB** (the `incidents/rca_evidence` tables) to retrieve past resolutions relevant to the current anomaly. This retrieved evidence is then passed into the Root-Cause and Planning agents' reasoning context — i.e., retrieval feeds generation, which is the core RAG pattern.
- This is combined with **lineage-context retrieval** (`get_lineage_context`) and **column/statistical retrieval** (`get_column_stats`, `get_sample_rows`), so the agent grounds its reasoning in retrieved structured evidence rather than relying purely on parametric LLM knowledge.
- **What the documents do not claim:** there is no mention of vector embeddings, semantic/similarity search, or a vector database. The retrieval described is structured/DB-query-based over DuckDB records, not embedding-based semantic RAG.

Recommended framing for your report: describe this as a "structured, tool-based retrieval-augmented reasoning mechanism" (evidence retrieval from DuckDB + lineage graph, injected into agent context before generation), and explicitly note that it is not a vector-embedding RAG pipeline — this keeps the deliverable honest for evaluators who will probe implementation depth.

5. MEMORY AND STATE MANAGEMENT

Two distinct memory/state layers exist:

a) In-process run state:

- `RUN_STATES` — an in-memory Python dictionary used to locate paused runs during approval. **Lost on backend restart** (explicitly flagged as a limitation in both documents).

b) LangGraph checkpointing:

- Uses **MemorySaver** to checkpoint graph state at the `human_review_queue` interrupt, keyed by `run_id`, enabling pause-and-resume across the approval boundary.
- Also explicitly flagged as volatile/in-memory — both documents recommend PostgreSQL (or SQLite) checkpointing for production/distributed deployment as future work.

c) Persistent historical memory (DuckDB):

- `runs`, `incidents`, `dq_results`, `rca_evidence`, `remediation_results`, `evaluation_records` — six tables giving durable, queryable memory of past pipeline executions, which is what `search_similar_incidents` draws on.

So: **short-term/execution state** = in-memory (`RUN_STATES` + `MemorySaver` checkpoint); **long-term/historical memory** = DuckDB persistence layer. This is a reasonable point to raise under "state management maturity" — the design correctly separates the two, but the short-term layer is not fault-tolerant.

6. PRELIMINARY EVALUATION RESULTS

Evaluation is split into two grounded levels, explicitly designed to avoid overclaiming LLM performance:

System contract validation:

- **49 Pytest test functions** (one doc cites 33; use 49 as the more recent figure from the Project Report) covering data loading, DQ detection, lineage traversal, graph routing, human-review interrupt/resume, SQL healing, remediation/quarantine, DuckDB persistence, API contracts, and evaluation-logic correctness (including a dedicated test ensuring fallback runs can't be miscounted as live-LLM successes).

Controlled incident benchmark (20 synthetic incidents, C01–C20):

- **Controlled Incident Handling Success Rate: 100.0% (20/20)**
- **Controlled Verification Consistency: 100.0%** — including a deliberately unresolvable date-corruption case (C05-type), where the system correctly reported VERIFICATION_FAILED instead of a false success.

Live-LLM (Groq) evaluation:

- 82 call failures due to external rate limiting during multi-turn interactions.
- Affected triage decisions correctly degraded to FLAG_FOR_REVIEW with confidence = 0.0, routing to human approval.
- Separately reported live-Groq run: **0.0% Agent Task Success Rate, 50.0% Tool Success Rate** — included specifically to demonstrate the system distinguishes deterministic workflow success from genuine live-LLM reasoning success.

Six grounding rules govern the evaluation (explicitly stated as a methodological contribution): (1) test pass rate $\neq$ LLM quality; (2) offline fallback success $\neq$ agent task success; (3) VERIFICATION_FAILED = correct behavior, not failure; (4) live metrics require successful non-fallback execution; (5) RCA precision/recall/F1 reported as **not measurable** (no annotated ground truth exists yet); (6) failure recovery rate reported as **not measurable** (no fault injection implemented yet).

This last point is worth calling out to reviewers directly — it's a strength (honest, ungamed evaluation) but also an admitted current gap (several claimed capabilities lack quantitative evidence yet).

7. COMPARISON WITH BASELINE SOLUTION

Directly from the Project Report's comparative table (Traditional Assertion Suites vs. Naive LLM Text-to-SQL Agents vs. ObsidianDQ):

Dimension	Traditional (Great Expectations, Soda)	Naive LLM Agents	ObsidianDQ
Anomaly Detection	Rigid assertion tests, manual config	Unbounded prompt-based scanning	Vectorized, deterministic Python rule engine
Root-Cause Investigation	None - static alerts to Slack/Email	Hallucinatory text explanation, no proof	Lineage DAG traversal + read-only tool calling
SQL Healing	None - manual refactoring	Unconstrained LLM rewrite (frequent syntax errors)	AST-grounded parsing via sqlglot
Warehouse Write Permissions	Read-only detection	Direct read/write credentials (critical risk)	Zero direct source writes; isolated quarantine
Human-in-the-Loop Governance	External ticketing (Jira)	Brittle prompt pauses / unassisted autonomy	Stateful graph checkpoint interrupts (MemorySaver)
Verification Cycle	Manual pipeline re-run	Assumed success on generation	Automated post-remediation verification
Adversarial Safety Audit	None	None	Dedicated Critic Agent, bounded retry (≤ 1)
Failure Recovery Mode	Hard pipeline failure	Unpredictable loop / silent corruption	Graceful fail-closed deterministic fallback

Core comparative argument for your report: traditional tools are safe but not investigative; naive LLM agents are investigative but operationally unsafe (direct write access, hallucination risk, no verification); ObsidianDQ's contribution is architectural - separating LLM reasoning (read-only investigation/planning/triage) from deterministic execution (detection, lineage, SQL AST transform, quarantine, verification) to get investigative capability without write-access risk.